

Paper

“latex

1 Summary

Bài báo *LoopFM: Learning frOm HistOrical RePresentations of Foundation Model for Recommendation* nghiên cứu bài toán truyền tri thức từ *Foundation Model* (FM) rất lớn sang các *Vertical Models* (VMs) nhỏ hơn trong hệ recommender công nghiệp. Trong thực tế, FM có thể có hàng nghìn tỷ tham số, được huấn luyện offline trên dữ liệu cross-domain rất lớn và rất giàu feature. Ngược lại, VM thường chỉ có hàng triệu tham số, được dùng để phục vụ online dưới ràng buộc latency nghiêm ngặt. Vì vậy, hệ thống cần một cơ chế để chuyển tri thức từ FM sang VM mà không yêu cầu chạy FM real-time trong serving.

Cách phổ biến hiện nay là *Knowledge Distillation* (KD). FM sinh ra một scalar prediction hoặc soft label $\hat{y}_F$, sau đó VM được huấn luyện để học theo soft label đó bên cạnh nhãn thật. Tuy nhiên, paper cho rằng scalar KD tạo ra một *bandwidth bottleneck*. Một FM lớn học được nhiều loại tri thức như cross-domain features, multi-level interactions, contextual patterns và user behavior signals, nhưng tất cả bị nén vào một con số duy nhất. Khi FM ngày càng lớn, lượng tri thức học được tăng lên, nhưng khả năng truyền qua một scalar gần như không tăng, làm *transfer ratio* giảm.

Paper định nghĩa *transfer ratio* là tỷ lệ cải thiện của FM mà VM hấp thụ được:

$$TR = \frac{\Delta NE_{VM}}{\Delta NE_{FM}}$$

Trong đó NE là *Normalized Entropy*, tức normalized LogLoss. Khi FM upgrade mạnh nhưng VM chỉ cải thiện ít, transfer ratio thấp. Bài báo quan sát rằng khi khoảng cách giữa FM và VM tăng lên, scalar KD ngày càng kém hiệu quả.

Để giải quyết vấn đề này, bài báo đề xuất *LoopFM*, một framework mở thêm kênh truyền tri thức có bandwidth cao hơn. Thay vì chỉ chuyển scalar prediction, LoopFM trích xuất intermediate embeddings từ FM, nén chúng lại, rồi tổ chức thành historical sequence để VM dùng như feature đầu vào. Điểm quan trọng là LoopFM chỉ dùng *historical embeddings*, nên khi serving online, VM không cần gọi FM real-time.

Pipeline của LoopFM gồm ba stage chính: *extraction*, *compression*, và *structuring*. Ở

stage thứ nhất, *embedding extraction*, với một FM có các layers $l_1, \dots, l_M$, hệ thống chọn một số layer và nối activation của chúng để tạo raw embedding:

$$e^{(i)} = [h_{l_1}^{(i)}; \dots; h_{l_K}^{(i)}] \in \mathbb{R}^D$$

Trong production, D có thể rất lớn, từ 10^5 đến 10^6 , vì FM có nhiều tầng và hidden representation lớn. Việc extraction có thể dùng chung inference pass với KD nên gần như không thêm overhead lớn trong offline logging.

Stage thứ hai là *compression*. Vì raw FM embeddings quá lớn để lưu trực tiếp, LoopFM dùng autoencoder để nén:

$$z^{(i)} = f_{enc}(e^{(i)})$$

$$\hat{e}^{(i)} = f_{dec}(z^{(i)})$$

$$L_{AE} = \|e^{(i)} - \hat{e}^{(i)}\|_2^2$$

Trong đó $z^{(i)} \in \mathbb{R}^d$ và $d \ll D$. Autoencoder được co-trained với FM nhưng dùng stop-gradient để không làm gradient từ autoencoder chảy ngược vào FM backbone. Paper cũng dùng *tanh* ở layer cuối của encoder để bound output trong $[-1, 1]$, giúp lượng tử hóa dễ hơn.

Một kỹ thuật quan trọng trong compression là *INT4 quantization*. Sau khi encoder tạo embedding z , hệ thống lượng tử hóa:

$$z_{quant} = \frac{\text{round}(8z).\text{clamp}(-8, 7)}{8}$$

Cách này giúp giảm storage khoảng $4\times$ so với FP16. Trong internal production, paper còn dùng K-means INT4, học 16 cluster centers không đều để phù hợp hơn với phân phối embedding, giúp giảm chất lượng mất mát so với vanilla INT4.

Paper cũng dùng *Matryoshka compression* để hỗ trợ nhiều kích thước embedding khác nhau trong deployment. Ý tưởng là mọi prefix $z_{1:d'}$ đều là một representation hợp lệ. Loss được tính trên nhiều target dimensions:

$$L_{MAE} = \sum_{d' \in \mathcal{D}} \|e - f_{dec}^{(d')}(z_{1:d'})\|_2^2$$

Nhờ đó, hệ thống có thể chọn dimension nhỏ hoặc lớn tùy trade-off giữa feature value, storage cost và latency cost.

Stage thứ ba là *structuring*. Các compressed embeddings được group theo một key, ví dụ user ID, item ID hoặc semantic ID. Trong paper này, tác giả tập trung vào user-keyed temporal sequence. Với user u , hệ thống lấy các historical embeddings trong một retention

window, ví dụ 30 ngày, rồi tạo sequence:

$$S_u = [z_{u,t_1}, z_{u,t_2}, \dots, z_{u,t_L}]$$

với:

$$t_L < \dots < t_2 < t_1 < t_{cur}$$

Trong đó t_{cur} là thời điểm serving hiện tại. Điểm quan trọng là sequence không chứa current sample, nhằm tránh yêu cầu real-time FM inference.

Ở phía VM, sequence S_u được đưa qua một sequence encoder như mean pooling, sum pooling, DIN attention hoặc DMIN attention. Representation thu được được concatenate với các feature embeddings khác của VM trước khi vào interaction layers. Loss huấn luyện VM kết hợp task loss và KD loss:

$$L = L_{task}(\hat{y}_V, y) + \lambda L_{KD}(\hat{y}_V, \hat{y}_F)$$

với:

$$\hat{y}_V = g(x_{VM}, S_u; \Theta_V)$$

Như vậy, LoopFM không thay thế KD mà bổ sung một kênh tri thức mới.

Về mặt lý thuyết, paper phân tích gain của LoopFM bằng mutual information. FM quan sát feature set rộng hơn:

$$x_{FM} = (x_{VM}, x_{extra})$$

trong đó x_{extra} là các feature mà VM không có. User history raw phía VM là:

$$H_u = (x_{VM}^{(t_1)}, \dots, x_{VM}^{(t_L)})$$

LoopFM sequence là:

$$S_u^{(k)} = [z_{u,t_1}^{(k)}, \dots, z_{u,t_L}^{(k)}]$$

Theorem 1 của paper phân rã information gain của LoopFM:

$$I_{LoopFM}(FM_k) = I_{temporal} + I_{cross,k} - I_{residual,k}$$

Trong đó:

$$I_{temporal} = I(H_u; y \mid x_{VM}^{(t)})$$

là thông tin dự đoán từ lịch sử user vượt ra ngoài current VM features.

Thành phần:

$$I_{cross,k} = I(S_u^{(k)}; y \mid x_{VM}^{(t)}, H_u)$$

là phần thông tin bổ sung do FM encode được từ các extra features hoặc cross-domain features mà VM không có.

Thành phần:

$$I_{residual,k} = I(H_u; y \mid x_{VM}^{(t)}, S_u^{(k)})$$

là phần thông tin còn sót lại trong raw history nhưng không được compressed sequence giữ lại, tức chi phí do representation loss, autoencoder loss và quantization loss.

Ý nghĩa của phân rã này là LoopFM có hai nguồn lợi chính: user temporal history và cross-feature knowledge từ FM. Nhưng gain cuối cùng bị trừ bởi compression loss. Vì vậy, chất lượng compression, embedding dimension và quantization đều ảnh hưởng trực tiếp đến hiệu quả chuyển tri thức.

Paper tiếp tục phân tích transfer ratio khi FM được nâng cấp từ FM_1 sang FM_2 . Giả sử:

$$x_{FM_1} = (x_{VM}, x_{extra,1})$$

$$x_{FM_2} = (x_{VM}, x_{extra,1}, x_{extra,2})$$

Tức FM_2 có thêm feature block so với FM_1 . Tổng improvement của teacher được viết:

$$\Delta_{teacher} = R_{ach}(FM_1) - R_{ach}(FM_2)$$

và phân rã thành:

$$\Delta_{teacher} = \underbrace{R^*(FM_1) - R^*(FM_2)}_{\Delta_{feat} \geq 0} + \underbrace{\epsilon_{over}(p_1, m_1) - \epsilon_{over}(p_2, m_2)}_{\Delta_{param}}$$

Trong đó Δ_{feat} là gain do feature phong phú hơn, còn Δ_{param} là gain do model capacity/overparameterization.

Theorem 2 đưa ra transfer-ratio lower bound cho LoopFM. Với feature gap:

$$\delta = m_2 - m_1$$

paper cho thấy lower bound có dạng:

$$TR_{LoopFM} \geq \frac{-\tau_2 I_{temporal} + (1 - \eta_1) \kappa_{gap}^{hist} \delta}{\bar{\kappa}_{gap} \delta + \bar{\kappa}_{over} \xi_1 - \kappa_{over} \xi_2}$$

Trong đó τ và η là các tham số biểu diễn pipeline loss; κ_{gap}^{hist} đo thông tin lịch sử từ feature gap; ξ_k liên quan đến excess risk của FM. Corollary 4 cho thấy bound này tăng đơn điệu theo feature gap δ , tức LoopFM càng có lợi khi FM mới có nhiều feature bổ sung mà VM không trực tiếp dùng được.

Theorem 5 phân tích ảnh hưởng của sequence length L . Paper cho rằng information gain của LoopFM:

$$I_{LoopFM,k}(L) = I(S_u^{(k,L)}; y \mid x_{VM}^{(t)})$$

không giảm khi tăng L và hội tụ đến một hằng số bị chặn bởi entropy:

$$I_{LoopFM,k}^* \leq H(y \mid x_{VM}^{(t)})$$

Điều này giải thích vì sao dùng sequence dài hơn thường cải thiện performance nhưng có diminishing returns.

Về thực nghiệm public benchmarks, paper dùng ba dataset: TaobaoAd, KuaiVideo và Amazon Electronics. TaobaoAd có 25M samples và 22 features, KuaiVideo có 13.7M samples và 9 features, Amazon Electronics có 3M samples và 6 features. Paper dùng DMIN làm FM và đánh giá nhiều VM như FM, FmFM, DeepFM, DIEN, DMR và DMIN.

Trên TaobaoAd, LoopFM cải thiện mạnh trên mọi VM. Ví dụ với DeepFM, baseline AUC là 0.5886, KD tăng lên 0.5980, LoopFM-only tăng lên 0.6245, và KD+LoopFM đạt 0.6344. Với DMIN VM, dù kiến trúc VM giống FM nên KD chỉ cải thiện ít, LoopFM vẫn đem lại gain lớn, cho thấy embedding sequence chứa tri thức bổ sung mà scalar KD không truyền được.

Paper cũng so sánh LoopFM với các embedding-based transfer methods khác. Các phương pháp như dùng current-sample FM embedding làm feature, FitNets-style current-embedding loss, hoặc user-only embedding đều gần như không cải thiện đáng kể so với KD. Trong khi đó, LoopFM đạt mức tăng lớn. Điều này cho thấy giá trị chính không nằm ở việc truyền một embedding hiện tại, mà nằm ở việc biến historical FM representations thành sequence feature.

Về sequence aggregation, paper cho thấy không phải chỉ attention mới có ích. Mean-pool historical embeddings đã cải thiện so với KD baseline. Sum-pool còn mạnh hơn đáng kể, đạt AUC 0.6300 trên TaobaoAd. DMIN attention đạt kết quả tốt nhất 0.6344. Điều này cho thấy ngay cả lưu trữ một pooled historical FM embedding cũng có thể là hướng feature engineering giá trị, còn sequence modeling tốt hơn giúp khai thác thêm temporal structure.

Paper cũng so sánh LoopFM với raw user sequences. Khi VM lớn và có đủ capacity, raw sequences đã rất mạnh và LoopFM chỉ nhỉnh hơn nhẹ. Nhưng khi VM nhỏ hơn 16 lần, LoopFM vượt raw sequences rõ rệt hơn. Điều này hợp lý vì VM nhỏ khó tự học embedding tốt từ high-cardinality raw IDs, trong khi LoopFM cung cấp representation đã được FM lớn học sẵn.

Phần hyperparameter analysis cho thấy layer selection rất quan trọng. Shallow layers thường transfer tốt hơn deep layers vì giữ nhiều thông tin đầu vào hơn, nhưng layer quá nông như embedding layer lại thiếu learned interactions. Hidden-0 hoặc joint compression nhiều layer thường tốt. Soft-label dưới dạng sequence 1 chiều cũng có gain đáng kể, cho thấy temporal structure của FM predictions tự nó đã chứa thông tin.

Một kết quả thú vị là FM checkpoint freshness không phải lúc nào cũng tốt hơn. Dùng checkpoint mới cho mỗi split làm performance giảm so với fixed checkpoint. Paper

giải thích bằng *embedding drift*: khi embeddings từ nhiều ngày khác nhau đến từ các checkpoint khác nhau, chúng không còn nằm trong cùng một representation space ổn định, khiến sequence encoder khó học pattern. Vì vậy, cần cân bằng giữa freshness và representation consistency.

Về sequence length, kết quả phù hợp với Theorem 5. AUC tăng từ 0.6327 với $L = 10$ lên 0.6352 với $L = 100$, nhưng phần lớn gain đã được capture ở $L = 50$. Về embedding dimension, nhờ Matryoshka autoencoder, AUC khá ổn định từ $d = 8$ đến $d = 128$ trên public benchmark. Tuy nhiên, trong internal production với FM lớn hơn nhiều, embedding dimension cao hơn cần đi kèm sequence encoder rộng hơn để khai thác hết thông tin.

Ở quy mô production, LoopFM được kiểm chứng trên trillion-parameter FMs và billion-scale daily examples. Kết quả cho thấy LoopFM gần như *double transfer ratio* so với KD alone, và KD + LoopFM thường tốt hơn từng kênh riêng lẻ, chứng minh hai kênh truyền tri thức này mang thông tin bổ sung. Online, LoopFM đem lại +0.5% conversion improvement trong Y1H1, và +1.03%, +1.22% conversion improvements từ hai launches riêng trong Y1H2.

2 Conclusion

Hướng đi đã làm của bài báo là mở rộng knowledge transfer trong recommender system từ scalar KD sang multi-channel transfer. Thay vì chỉ truyền prediction score của FM cho VM, LoopFM trích xuất intermediate FM embeddings, nén chúng, lượng tử hóa nếu cần, rồi tổ chức thành historical sequences để VM dùng trực tiếp như feature. Cách này mở ra một kênh truyền tri thức có bandwidth cao hơn, trong khi vẫn không yêu cầu real-time FM inference khi serving.

Đóng góp kỹ thuật quan trọng nhất là framework ba stage: *extraction*, *compression*, và *structuring*. Extraction chọn layer activation từ FM; compression dùng autoencoder, Matryoshka representation và quantization để giảm storage; structuring nhóm embeddings theo key, đặc biệt là user-keyed temporal sequence. Thiết kế này modular, nên từng stage có thể được cải tiến độc lập.

Về lý thuyết, paper đóng góp gain decomposition:

$$I_{LoopFM} = I_{temporal} + I_{cross} - I_{residual}$$

Công thức này giải thích rõ LoopFM có lợi vì nó vừa khai thác temporal history, vừa truyền cross-feature knowledge từ FM, nhưng bị giới hạn bởi compression loss. Transfer-ratio analysis cũng cho thấy LoopFM đặc biệt hữu ích khi FM có feature gap lớn so với VM, tức một tình huống rất phổ biến trong production recommender.

Kết quả thực nghiệm cho thấy LoopFM bổ sung tốt cho KD chứ không thay thế KD. KD truyền thông tin về current prediction, còn LoopFM truyền historical representation

và cross-feature interactions. Trên TaobaoAd, KD+LoopFM đạt AUC cao nhất ở hầu hết mô hình. Trên KuaiVideo và Amazon, gain nhỏ hơn nhưng vẫn nhất quán, đặc biệt khi VM thiếu sequence features hoặc có capacity hạn chế.

Ý nghĩa thực tiễn là trong hệ FM-VM công nghiệp, việc chỉ dùng scalar KD có thể bỏ lỡ phần lớn tri thức của FM. Khi FM ngày càng lớn và dùng nhiều cross-domain features hơn, cần truyền intermediate representations hoặc structured historical features để VM hấp thụ nhiều hơn. LoopFM cho thấy cách làm này có thể triển khai production với quantization và không làm tăng serving dependency vào FM.

Hướng kế thừa từ bài báo có thể gồm nhiều nhánh. Thứ nhất, có thể mở rộng structuring từ user-keyed temporal sequences sang item-keyed sequences, semantic-keyed sequences hoặc graph-based structures như user-item bipartite graph với FM embeddings trên edges. Thứ hai, có thể cải thiện compression bằng residual quantization, vector quantization, learned codebook hoặc contrastive compression để giảm $I_{residual}$. Thứ ba, có thể nghiên cứu self-LoopFM, tức FM tự tiêu thụ historical embeddings của chính nó để tạo vòng lặp tự cải thiện representation.

Ngoài ra, paper này rất phù hợp để kế thừa cho hướng recommender system có nhiều context và long user history. Các FM lớn thường học được rich context interactions nhưng VM phục vụ online bị giới hạn feature và latency. LoopFM cung cấp một cơ chế biến tri thức offline của FM thành feature online rẻ hơn cho VM. Một hướng phát triển hợp lý là kết hợp LoopFM với time-aware hoặc context-aware recommendation, trong đó historical FM embeddings không chỉ được dùng như sequence thông thường mà còn được tổ chức theo thời gian, session, domain, intent hoặc persona để tăng khả năng cá nhân hóa. ““